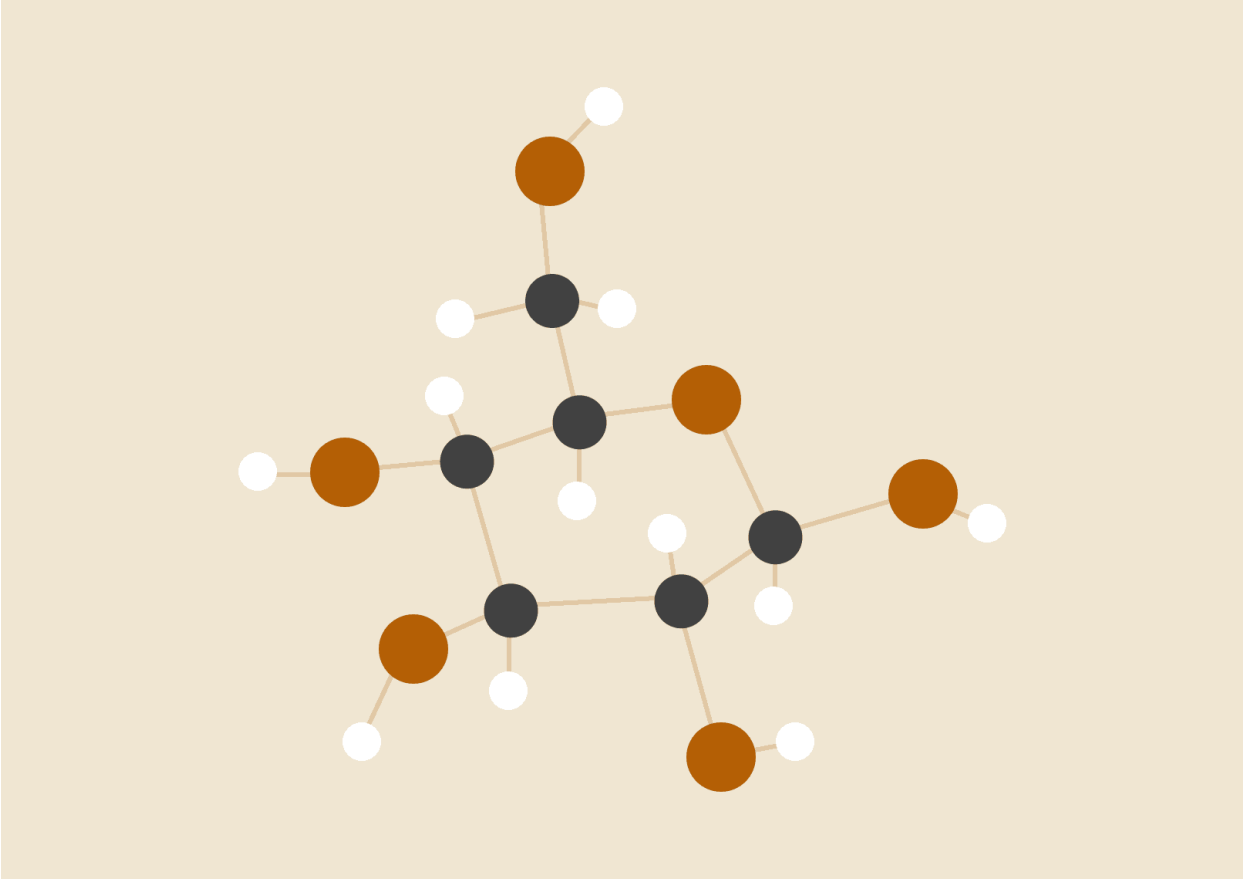
REPORT ON MATRIX MULTIPLICATION

Parallel architectures



Jaumotte Hugo

June 8, 2026

HEH be
Sciences
et technologies

SUMMARY

INTRODUCTION

The aim of this report is to run, on the same machine, several codes to multiply matrices: sequential, multithreaded and CUDA? This will allow us to compare the performances of the different versions and see which one is the best for this type of simple problem.

MACHINE CONFIGURATION

The following table describe the characteristics of the PC used to run the different codes:

Operating System	Microsoft Windows 11 Home (Build 26200)
Machine	ASUS TUF Dash F15 FX517ZE
CPU	Intel Core i7-12650H @ 2.30 GHz
CPU Configuration	10 physical cores / 16 logical threads
Memory	16 GB RAM, DDR5-4800
GPU	NVIDIA GeForce RTX 3050 Ti Laptop GPU
GPU Memory	4 GB GDDR6
Storage	Intel SSDPEKNU512GZ NVMe SSD, 512 GB
NVIDIA Driver	610.47
CUDA Version	13.3

PART 1: SEQUENTIAL

Here are the time executions of the sequential code in µs for different sizes of matrices.

N (Matrice size: NxN)	Execution time (µs)
100	2 185
500	264 608
1000	2 170 507
1500	8 416 846
2000	25 201 809

We can clearly see an exponential increase of the execution time, which is normal since an increase of N as both an increase on line and column number of the matrix. The execution time becomes quite consequent for the last itération : 25 sec for N = 2000, so 4 000 000 cells.

PART 2: ONE THREAD PER CELL

For the first multithreaded code, we decide to implement a solution with one thread per cell, it is non optimal for sure, but give us a first comparison point for the next implementations.

N (Matrice size: NxN)	Execution time (µs)
100	371 696
500	51 284 456
1000	—
1500	—
2000	—

As we can see, this first test with the threads is less efficient than the previous sequential code. This shows us that just applying multithreading without thinking of the problem characteristics or machine specificity does not work. For the 3 last lines, the machine was

not able to produce a result, the execution led to a frozen screen and an excessive warmth. This is due to the excessive number of threads, but also the age of the machine.

PART 3: ONE THREAD PER ROW

In this implementation, we drop the number of threads significantly by giving more work to each of them. For a matrix of N columns and N lines the number of threads is now equal to N instead of N^2 in the previous section, which led to a division of N in the number of threads. Knowing that the high-execution times of the previous versions were probably due to the high number of threads, this version might be more effective.

N (Matrice size: NxN)	Execution time (µs)
100	9 873
500	47 051
1000	243 934
1500	1 261 800
2000	3 931 811

We see that for the small values, the sequential code is still better that this one, but as soon as the number of lines and columns becomes consequent (somewhere between n=100 and N=500), the execution time is shorter for this version of the code.

PART 4

The number of thread used for each execution is equal to :

0.25T, 0.5T, 0.75T, T, 1.5T or 2T

Where T is the number of threads of the CPU.

PART 4A: ALTERING THREAD DISTRIBUTION

In this version, for each adjacent cell, we use a different thread, in order to have a repartition similar to this :

T0	T1	T2	T3	T0
T1	T2	T3	T0	T1
T2	T3	T0	T1	T2
T3	T0	T1	T2	T3
T0	T1	T2	T3	T0

Table of the execution times in microseconds

N (Matrice size: NxN)	4 threads	8 threads	12 threads	16 threads	24 threads	32 threads
100	2 107	2 604	3 415	4 319	5 151	7 447
500	70 794	41 174	46 884	42 951	34 781	36 087
1000	512 042	319 530	298 655	243 844	259 105	236 356
1500	1 736 629	1 046 745	1 275 455	1 217 426	1 256 000	1 210 091
2000	4 403 327	2 799 776	3 230 946	4 150 859	3 556 243	3 591 684

As we can see in the results, when the number of lines and columns start to become consequent (1500+), the best results in terms of execution time is with 8 threads (0.5xT). If we run the code with more threads , the execution times are higher. The results obtained for 8 threads are better in this version of the code than the previous one.

PART 4B: CONTIGUOUS THREAD DISTRIBUTION

In this version, we divide the number of cells by the number of threads to know if the number of cells calculated per thread. Each cell of the same thread is contiguous.

T0	T0	T0	T0	T0
T0	T1	T1	T1	T1
T1	T1	T2	T2	T2
T2	T2	T2	T3	T3
T3	T3	T3	T3	T3

Table of the execution times in microseconds

N (Matrice size: NxN)	4 threads	8 threads	12 threads	16 threads	24 threads	32 threads
100	1 409	3 006	4 068	4 577	4 617	3 145
500	82 829	43 989	50 057	40 409	32 764	35 720

1000	515 512	335 043	254 117	244 172	256 683	226 951
1500	1 766 222	1 018 748	1 056 701	1 273 349	1 318 831	1 216 211
2000	4 631 728	3 075 235	3 384 295	3 985 669	3 624 020	3 618 770

Just like the part 4A, the best solution is to set the number of threads to 8 (0.5xT). I expected the results to be better in terms of execution time, but the results are less good. Overall, the difference does not seem to be significant.

PART 5: CUDA

This is the only section in which cells of the matrix are calculated by the GPU. The number of blocks and threads will vary in the différent executions.

Table of the execution times in microseconds (without transfers)

N (Matrice size: NxN)	128 blocs 128 threads	256 blocs 128 threads	128 blocs 256 threads	512 blocs 256 threads
100	1 083	1 072	838	848
500	1 434	1 761	1 803	1 953
1000	6 299	7 262	11 739	75 093
1500	21 014	58 691	54 921	52 619
2000	46 606	94 570	80 111	111 914

With transfers :

N (Matrice size: NxN)	128 blocs 128 threads	256 blocs 128 threads	128 blocs 256 threads	512 blocs 256 threads
100	1 661	1 747	1 088	1 160
500	2 197	2 856	2 801	2 852
1000	9 617	9 819	14 885	82 934
1500	27 254	67 459	60 833	57 663
2000	56 058	106 642	90 993	120 792

The implementation of the solution using GPU for calculation is more difficult because it highly depends on the problem or the machine, but the execution time seems to be significantly smaller than the execution via CPU.

For this section, I ran the code using différent configurations in terms of blocks and threads number. The results obtained show that the version with 128 blocks and 128 threads per block is the most efficient with around 0.06 sec for a 2000x2000 matrix calculation or bigger.

CONCLUSION

The following table is the resume of the most effectives résults for each part :

N (Matrice size: NxN)	Part 1	Part 2	Part 3	Part 4A (8 threads)	Part 4B (8 threads)	Part 5 128x128
100	2 185	371 696	9 873	2 604	3 006	1 661
500	264 608	51 284 456	47 051	41 174	43 989	2 197
1000	2 170 507	—	243 934	319 530	335 043	9 617
1500	8 416 846	—	1 261 800	1 046 745	1 018 748	27 254
2000	25 201 809	—	3 931 811	2 799 776	3 075 235	56 058

To conclude, the utilisation of multithreaded code can be a huge tool to reduce the run time if implemented correctly. As we have seen in the second part, an implementation that does not consider the initial problemùen correctly can lead to results that are worse than a sequential code.

Then, by considering the results obtained in the parts 3, 4A and 4B, we can see that we were able to reduce the execution time for around 25 to 3 sec for a 2000x2000 matrix, which will lead to even bigger time gains as the matrix grows up.

Finally, the implementation of the solution in CUDA, to run the calculation on the GPU allows us to reduce the run time of the 2000x2000 matrix even further, to 0.06 seconds, which stands for a 99.78% time reduction compared to the part 1.